

Voici une documentation complète de l'architecture des applications développer par la CPD, expliquant la logique, la structure et les interactions entre les fichiers. Cette documentation est structurée pour être claire et facile à partager avec l'ensemble de l'équipe.

DOCUMENTATION DE L'ARCHITECTURE FRONTEND (REACT/TYPESCRIPT)

Objectif : Ce document a pour but d'expliquer la logique derrière l'organisation de notre code, le rôle de chaque dossier/fichier clé, et comment ils interagissent. L'objectif est de maintenir une base de code cohérente, maintenable et compréhensible par toute l'équipe.

1. Philosophie Générale : Organisation par Fonctionnalité

Nous organisons le code par **fonctionnalité** (feature) plutôt que par type de fichier.

- **Pourquoi ?** Il est plus facile de retrouver tous les fichiers liés à une fonctionnalité (ex : authentification) au même endroit. Si nous devons modifier ou déboguer le processus d'inscription, nous savons que tout se trouve dans `src/api/registration/`. Cela rend le code plus **maintenable** et **évolutif**.
- **Principe Clé :** Chaque fichier a une responsabilité unique et claire. Cela évite les duplications et les "effets de bord" inattendus.

2. Structure du Code et Rôle des Dossiers

Voici l'arborescence des dossiers et le rôle de chacun :

```
src/
├─ api/                # Toute la communication avec le backend
│  ├─ axios.ts         # Configuration et intercepteurs Axios (coeur du réseau)
│  ├─ types.ts         # Types TypeScript globaux pour les réponses API
│  ├─ authentication/  # Logique de connexion et 2FA
│  ├─ registration/   # Logique d'inscription et validation OTP
│  └─ resetPassword/   # Logique de réinitialisation de mot de passe
├─ components/         # Composants d'interface réutilisables (UI)
│  ├─ auth/            # Composants liés à l'auth (ex : RequireAuth)
│  ├─ theme/           # Composants pour la gestion du thème
│  └─ ui/              # Composants de base (shadcn/ui) : boutons, inputs, etc.
├─ hooks/              # Logique métier réutilisable (Custom Hooks)
│  ├─ useAutoLogout.ts  # Gère la déconnexion pour inactivité
│  ├─ useConnectedUser.ts # Fournit les infos de l'utilisateur connecté
│  └─ useHeartbeat.ts   # Maintient la session active côté serveur
├─ pages/              # Les pages de l'application (points d'entrée des routes)
│  ├─ Authentication.tsx
│  ├─ Dashboard.tsx
│  └─ ... (toutes les autres pages)
├─ utils/              # Fonctions utilitaires génériques (sans JSX)
│  └─ jwt.ts           # Fonctions pour parser et valider les tokens JWT
└─ App.tsx            # Point d'entrée : configuration des routes et providers
```

3. Explication Détaillée des Fichiers Clés et de leurs Interactions

3.1. Le Coeur du Réseau : src/api/axios.ts (et types.ts)

- `src/api/types.ts` : Définit le contrat de réponse standard de notre API.

```
// Pourquoi ? Pour garantir que toutes les réponses API ont la même structure.  
// Cela permet de typer les données de manière fiable dans toute l'appli.  
export interface ApiResponse<T> {  
  success: boolean;  
  code: number;  
  title: string;  
  message: string;  
  data: T; // La partie variable de la réponse, typée par T.  
}
```

- `src/api/axios.ts` : C'est le **cerveau de la communication réseau**. Il est centralisé pour une raison simple : **ne pas répéter la logique de gestion des tokens et des erreurs** dans chaque appel API.
 - **Configuration** : Crée une instance Axios avec une `baseUrl` et des `headers` par défaut, lue depuis les variables d'environnement. Tous les appels API passeront par cette instance.
 - **Intercepteur de Requête (Nouvelle version)** :
 - Avant chaque requête, il vérifie le token.
 - Si le token existe **et expire bientôt** (grâce à `isTokenExpiringSoon` de `jwt.ts`), il tente de le rafraîchir **préventivement** en appelant `refreshToken()`.
 - Cela évite d'arriver à une requête avec un token expiré, améliorant ainsi l'expérience utilisateur.
 - **Fonction `refreshToken()`** :
 - Contient la logique de rafraîchissement. Utilise un système de "subscribers" pour éviter de multiples appels de refresh en parallèle.
 - **Point d'attention** : Elle met à jour non seulement le `token` mais aussi les infos utilisateur (`user`) dans le `localStorage`. Cela garantit que les données utilisateur sont toujours fraîches.
 - **Intercepteur de Réponse** :

- Intercepte les erreurs de réponse.
- Si une erreur **401 (Non autorisé)** se produit, il appelle `refreshToken()` et réessaie la requête originale. C'est le mécanisme de secours si le refresh préventif n'a pas eu lieu ou a échoué.
- En cas d'échec du refresh, il déconnecte l'utilisateur.

3.2. La Logique Métier Réutilisable : Les Hooks (`src/hooks/`)

Les hooks isolent la logique complexe des composants UI.

- `src/hooks/useConnectedUser.ts` :
 - **Pourquoi ?** Pour éviter de parser le `localStorage` et de recalculer les dérivées (comme `avatarInitials`) dans chaque composant qui a besoin des infos utilisateur.
 - **Fonctionnement** : Il lit l'objet `user` du `localStorage` (mis à jour par `axios.ts` lors du `login/refresh`), le parse et retourne un objet prêt à l'emploi avec toutes les informations nécessaires.
- `src/hooks/useHeartbeat.ts` :
 - **Pourquoi ?** Pour garder la session utilisateur active côté serveur sans nécessiter d'action de sa part. C'est un complément au refresh token.
 - **Fonctionnement** : À intervalles réguliers, il vérifie si le token expire bientôt. Si c'est le cas, il fait un appel silencieux (`/heartbeat`) pour "réveiller" la session et maintenir le token valide plus longtemps. La logique de rafraîchissement du token lui-même reste dans `axios.ts`.
- `src/hooks/useAutoLogout.ts` :
 - **Pourquoi ?** Pour des raisons de sécurité, on déconnecte l'utilisateur après une période d'inactivité sur l'interface.
 - **Fonctionnement** : Il écoute les événements utilisateur (souris, clavier). Si aucun événement n'est détecté pendant un certain temps, il appelle `logoutApi` et redirige vers la page de connexion.

3.3. Les Utilitaires Génériques : `src/utils/jwt.ts`

- **Pourquoi ?** Toute la logique de manipulation des tokens JWT est concentrée ici. Elle est pure (pas d'effet de bord) et donc facile à tester.
- **Fonctions :**
 - `parseJwt(token)`: Décode le token pour récupérer son contenu (payload).
 - `isTokenExpired(token)`: Vérifie si la date d'expiration est dépassée.
 - `isTokenExpiringSoon(token, seuil)`: Vérifie si le token va expirer dans un futur proche. Utilisée par `axios.ts` pour le refresh préventif.

3.4. Les Appels API Spécifiques (`src/api/authentication/*.api.ts`, etc.)

- **Pourquoi ?** Chaque fichier regroupe tous les appels liés à une même fonctionnalité (login, logout, etc.). Cela permet d'avoir une vision claire de toutes les interactions réseau pour un domaine donné.
- **Fonctionnement :**
 - Chaque fonction (ex : `loginApi`) importe l'instance `api` depuis `axios.ts` pour faire sa requête.
 - Elle définit son propre **payload** et son **type de réponse**, en utilisant le générique `ApiResponse<T>`.
 - **Principe :** Ces fonctions ne font que l'appel API et retournent la réponse. Elles ne gèrent ni les erreurs globales, ni le stockage du token. C'est le rôle de `axios.ts`.

3.5. La Protection des Routes : `src/components/auth/RequireAuth.tsx`

- **Pourquoi ?** C'est le gardien des routes privées. Il centralise la logique de vérification d'authentification.
- **Fonctionnement :**
 - Il est placé comme un wrapper autour des composants de page dans `App.tsx` (`<Route path="/dashboard" element={<RequireAuth><Dashboard /></RequireAuth>}</>`).
 - À chaque navigation, il vérifie la présence et la validité du token via `isTokenExpired()`.
 - Si l'utilisateur n'est pas authentifié, il est redirigé vers `/connexion` et les données de session sont nettoyées.
 - **C'est ici que les hooks de session sont activés** pour les pages protégées :

- `useAutoLogout()` est appelé pour surveiller l'inactivité.
- `useHeartbeat()` est appelé pour maintenir la session.
- Cela garantit que ces mécanismes ne tournent que lorsque l'utilisateur est connecté, économisant des ressources.

3.6. L'Orchestration : `src/App.tsx`

- **Pourquoi ?** C'est le point d'entrée de l'application. Il configure le routeur, les providers globaux (Thème, QueryClient, Helmet) et définit la structure de toutes les routes.
- **Logique :**
 - Il utilise `React.lazy` pour le chargement différé (code-splitting) des pages moins critiques, améliorant les performances initiales.
 - Il définit clairement quelles routes sont publiques et lesquelles sont protégées par `<RequireAuth>`.

4. Récapitulatif : L'Enchaînement Logique pour l'Authentification

Pour illustrer l'interaction, voici le cycle de vie d'une session utilisateur typique :

- **Connexion** : L'utilisateur se connecte via `Authentication.tsx` -> appelle `loginApi()`.
- **Stockage** : `axios.ts` reçoit la réponse, et les intercepteurs ne font rien de spécial ici. Le composant qui a appelé `loginApi` stocke le `token` et les infos `user` dans le `localStorage`.
- **Navigation vers une page protégée** : L'utilisateur va sur `/dashboard`.
 - `RequireAuth` vérifie le token (`isTokenExpired`).
 - Si valide, il active `useHeartbeat` et `useAutoLogout`.
- **Requête API (ex : chargement du dashboard)** : Le composant `Dashboard` utilise `useConnectedUser` pour afficher le nom, et fait un appel API.
 - **Avant l'envoi** : L'intercepteur de requête de `axios.ts` voit que le token expire bientôt. Il appelle `refreshToken()` silencieusement, met à jour le `localStorage` (token + user), puis envoie la requête originale avec le nouveau token. L'utilisateur ne voit rien.
 - **En cas de 401** : Si le refresh préventif a échoué ou n'a pas eu lieu, l'intercepteur de réponse reçoit une 401. Il tente un refresh, puis rejoue la requête.
- **Inactivité** : L'utilisateur laisse son ordinateur. `useAutoLogout` déclenche la déconnexion après 14 minutes, le nettoyant du `localStorage` et le redirigeant.
- **Fermeture de l'onglet** : La session est perdue. À la réouverture, `RequireAuth` ne trouvera pas de token et redirigera vers la connexion.

5. Pourquoi cette structure est robuste et maintenable

- **Séparation des préoccupations (SoC)** : Chaque fichier a un rôle précis. Les hooks ne connaissent pas l'UI, les appels API sont juste des fonctions, `axios.ts` gère le réseau, etc.
- **Testabilité** : Les fonctions pures (`jwt.ts`) et les hooks (avec des mocks) sont faciles à tester unitairement.
- **Évolutivité** : Pour ajouter une nouvelle fonctionnalité (ex : "Messages Privés"), il suffit de créer un dossier `api/messages/` avec ses appels, et d'utiliser les hooks existants pour l'utilisateur.
- **Évite la duplication** : La logique de refresh token, de parsing JWT, ou de récupération de l'utilisateur n'est écrite qu'une seule fois et réutilisée partout.

En suivant cette structure, nous nous assurons que le projet reste compréhensible et maintenable, même avec une équipe grandissante.